Rust Tutorial

Rust

Rust is a popular programming language used to build everything from web servers to game engines.

👉 In simple words: Rust is a tool programmers use to build powerful software.

For example:

- A **web server** (like the backend of a website that handles user requests)
- A **game engine** (the core system that runs a video game)

Rust is known for being very fast, and is similar to C and C++ in language.
👉 This means it runs programs very quickly, and its writing style looks similar to C and C++.

What is Rust?

Rust is a modern programming language.

👉 "Modern" means it is designed for today's needs — fast computers, large applications, and secure systems.

It is designed for:

- **Performance** (runs very fast)
- **Reliability** (doesn't crash easily)
- **Safety** — especially safe handling of memory

👉 Memory is the computer's working space (RAM). Rust makes sure that space is used safely and correctly.

Example:
If a program tries to use memory that doesn't belong to it, Rust will stop it before it causes a crash.

Why Learn Rust?

• It is one of the fastest-growing programming languages in the world
👉 More companies are starting to use it.

• Learning Rust makes it easier to pick up other languages like Java, Python, C++, and C#, because the syntax is similar.
👉 If you understand Rust, learning those languages becomes easier.

• Rust is very fast
👉 Good for apps where speed matters (like games or servers).

• Rust requires less memory compared to many other languages
👉 It doesn't waste system resources.

• Rust is used to build web servers, create games, operating systems, and much more!
👉 It can be used for both small programs and very large systems.

Difference Between Rust and C / C++

• The main difference is that Rust is "safe by default".

👉 This means Rust automatically protects you from certain common mistakes.

• In Rust, memory access is automatically checked by the compiler.
👉 The compiler checks your code before it runs and prevents common bugs like:

- Crashes
- Memory leaks
- Unsafe memory behavior

Example:
If you accidentally try to use memory after deleting it, Rust will give an error before the program runs.

• In C and C++, you must manually manage memory.
👉 The programmer has to handle memory themselves.
If they make a small mistake, it can cause serious errors.

Example:
Forgetting to free memory in C/C++ can cause a memory leak.
Rust helps prevent this automatically.

(REFER EXERCISE -1)

Install Rust

However, if you want to download and install rust, you can go to rust-lang.org and follow the instructions there.

Check Installation

After installing, check if Rust is installed correctly by running:

rustc --version

The output will look something like this, depending on your version number:

rustc 1.86.0 (05f9846f8 2025-03-31)

Create a New Project

Use Cargo to create a new Rust project:

```
cargo new my_project
```

The output will look something like this:

```
    Creating binary (application) `my_project` package
note: see more `Cargo.toml` keys and their definitions at
https://doc.rust-lang.org/cargo/reference/manifest.html
```

This creates a folder called `my_project` with the following files:

- `Cargo.toml`: Project settings
- `src/main.rs`: Main Rust file

The `main.rs` file contains this default code:

```
fn main() {
  println!("Hello, world!");
}
```

Build and Run the Project

Next, write the following code to go into the project folder:

```
cd my_project
```

Then build and run the project:

```
cargo run
```

The output should be:

```
Hello, world!
```

Line 1: fn main() is something that always appears at the beginning of every Rust program. main() is called a **function**, and any code inside its curly brackets {} will be executed.

Line 2: println!() is a **macro**, used to output/print text to the screen. In our example it will output "Hello World!". To end the code, you must remember a semicolon (;).

(**What is a macro?**

A macro is like a function, but with an exclamation mark (!) after it. Don't worry about the terminology for now. For now, just know that macros are similar to functions (they execute things), but they do not always follow the same rules as functions. You will learn more about macros later.(Note that it will add a new line for each macro)

Good to know: The Rust compiler ignores white spaces.

The code above could also been written as:
fn main(){println!("Hello World!");}

However, multiple lines and indentation makes the code more readable.)

```rust
fn main() {
  println!("Hello World!");
  println!("I am learning Rust.");
  println!("It is awesome!");
}
```

```
Hello World!
I am learning Rust.
It is awesome!
```

There is also a print!() macro, which is similar to println!().

The only difference is that it does not insert a new line at the end of the output:

If you really want to add a new line in print!(), you can use the \n character

```rust
fn main() {
  print!("Hello World! ");
  print!("I will print on the same line.");
}
```

```
Hello World! I will print on the same line.
```

```rust
fn main() {
  println!("Hello World!\nThis line was
broken up!");
}
```

```
Hello World!
This line was broken up!
```

Comments in Rust

Comments can be used to explain code, and to make it more readable. It can also be used to prevent execution when testing alternative code.

Single-line Comments

Single-line comments start with two forward slashes (`//`).

Any text between `//` and the end of the line is ignored by the compiler (will not be executed).

```
fn main() {
  // This is a comment
  println!("Hello World!");
}
```

```
Hello World!
```

Multi-line Comments

Multi-line comments start with `/*` and ends with `*/`.

Any text between `/*` and `*/` will be ignored by the compiler:

```
fn main() {
  /* The code below will print the
words Hello World!
  to the screen, and it is amazing */
  println!("Hello World!");
}
```

```
Hello World!
```

Variables

Variables are containers for storing data values, like numbers and characters.

To create a variable in Rust, use the `let` keyword and specify the name of the variable

(REFER TO EXERCISE-2)

```
println!("My first name is: {}", name)
```

(What is {}?

Rust uses {} as a placeholder in println!() to show variable values.)

Variable Values Cannot be Changed by Default

Data Types

Unlike many other programming languages, variables in Rust do not need to be declared with a specified type (like "String" for text or "Int" for numbers, if you are familiar with those from C or Java).

In Rust, the type of a variable is decided by the value you give it. Rust looks at the value and automatically chooses the right type:

(REFER EXERRCISE-3)

- **Numbers** - Whole numbers and decimal numbers (i32, f64)
- **Characters** - Single letters or symbols (char)
- **Strings** - Text, a sequence of characters (&str)
- **Booleans** - True or false values (bool)

Constants

Constant variables are used to store values that never change.

Unlike regular variables, constants must be defined with a **type** (e.g. `i32` or `char`).

Creating a Constant

To create a constant, use the `const` keyword, followed by the name, type, and value:

```
fn main() {
  const BIRTHYEAR: i32 = 2006;
  const MINUTES_PER_HOUR: i32 = 60;

  println!("What is your birth year?
{}", BIRTHYEAR);
  println!("How many minutes per hour?
{}", MINUTES_PER_HOUR);
}
```

```
What is your birth year? 1980
How many minutes per hour? 60
```

```rust
const BIRTHYEAR = 1980; // Error: missing type
```

Naming Rules

Another thing about constants, is that it is considered good practice to declare them with uppercase.

It is not required, but useful for code readability and common for Rust programmers:

Examples:

- MAX_SPEED
- PI
- MINUTES_PER_HOUR

Feature	Constant (`const`)	Variable (`let`)
Can change?	No	Yes, if `mut` is used
Type required?	Yes	No (optional)

Operators

Operators are used to perform operations on values and variables.

Rust supports many common operators, like:

- Arithmetic Operators
- Assignment Operators
- Comparison Operators
- Logical Operators

Arithmetic Operators

Arithmetic operators are used to do basic math:

Operator	Name	Example	Result
+	Addition	`5 + 3`	8
-	Subtraction	`5 - 3`	2
*	Multiplication	`5 * 3`	15
/	Division	`10 / 2`	5
%	Remainder (modulus)	`10 % 3`	1

Assignment Operators

Assignment operators are used to assign and update values:

Operator	Example	Same As
=	`x = 5`	Assign 5 to x
+=	`x += 3`	`x = x + 3`
-=	`x -= 2`	`x = x - 2`
*=	`x *= 4`	`x = x * 4`
/=	`x /= 2`	`x = x / 2`
%=	`x %= 2`	`x = x % 2`

Comparison Operators

Comparison operators compare values and return `true` or `false`:

Operator	Meaning	Example
==	Equal to	`5 == 5` is true
!=	Not equal to	`5 != 3` is true
>	Greater than	`7 > 3` is true
<	Less than	`2 < 5` is true
>=	Greater than or equal to	`5 >= 5` is true
<=	Less than or equal to	`3 <= 4` is true

Logical Operators

Logical operators are used to work with boolean values:

Operator	Name	Description
&&	AND	true if both values are true
||	OR	true if at least one is true
!	NOT	inverts the boolean value

(REFER EXERCISE-4)

◈ What is a Boolean?

A **boolean (`bool`)** is a data type that can only store:

- `true`
- `false`

That’s it. Only two values.

We use booleans when something has **two possible states**, like:

- YES / NO
- ON / OFF
- TRUE / FALSE
- Logged In / Not Logged In

◈ Creating Boolean Variables

✅ Method 1: Explicitly mention `bool`

```
let is_programming_fun: bool = true;
let is_fish_tasty: bool = false;

println!("Is Programming Fun? {}", is_programming_fun);
println!("Is Fish Tasty? {}", is_fish_tasty);
```

Here:

- `is_programming_fun` stores `true`
- `is_fish_tasty` stores `false`

✅ Method 2: Let Rust infer the type (more common)

Rust is smart. If you assign `true` or `false`, it automatically knows it’s a boolean.

```
let is_programming_fun = true;
let is_fish_tasty = false;
```

You don’t need to write `: bool`.

◈ Boolean from Comparison (Most Important 🔥)

In real programs, we usually don’t manually write `true` or `false`.

Instead, booleans come from **comparisons**.

Example:

```
let age = 20;
let can_vote = age >= 18;
```

```
println!("Can vote? {}", can_vote);
```

Here:

- `age >= 18` is a comparison.
- It returns `true` if the condition is correct.
- It returns `false` otherwise.

Since `20 >= 18` is true → `can_vote = true`

◈ Comparison Operators That Return Boolean

Operator	Meaning
`==`	Equal to
`!=`	Not equal to
`>`	Greater than
`<`	Less than
`>=`	Greater than or equal
`<=`	Less than or equal

Example:

```
let x = 10;
println!("{}", x == 10);   // true
println!("{}", x > 20);    // false
```

◈ Using Booleans in `if` Statements

Booleans are mainly used in decision making.

```
let is_logged_in = true;

if is_logged_in {
    println!("Welcome back!");
} else {
    println!("Please log in.");
}
```

If `is_logged_in` is:

- `true` → first block runs
- `false` → `else` block runs

◈ Important Rule in Rust 🚨

Unlike some other languages, Rust **does NOT allow numbers as booleans**.

❌ This is invalid:

```
if 1 {
}
```

✓ You must use a boolean expression:

```
if x > 0 {
}
```

Rust is strict — and that's good because it prevents bugs.

Conditions and If..Else

You already know that Rust supports familiar comparison conditions from mathematics, such as:

- Less than: `a < b`
- Less than or equal to: `a <= b`
- Greater than: `a > b`
- Greater than or equal to: `a >= b`
- Equal to `a == b`
- Not Equal to: `a != b`

You can use these conditions to perform different actions for different decisions.

Rust has the following conditional statements:

- Use `if` to specify a block of code to be executed, if a specified condition is `true`
- Use `else` to specify a block of code to be executed, if the same condition is `false`
- Use `else if` to specify a new condition to test, if the first condition is `false`
- Use `match` to specify many alternative blocks of code to be executed

Note: Unlike many other programming languages, `if..else` can be used as a statement or as an expression (to assign a value to a variable) in Rust. See an example at the bottom of the page to better understand it.

`if`

Use `if` to specify a block of code to be executed if a condition is `true`.

`if...else`

If the condition is not true, you can use `else` to run different code

`else if`

You can check multiple conditions using `else if`

Using `if` as an Expression

In Rust, `if...else` can also be used as an **expression**.

This means you can assign the result of an `if` to a variable:

When using `if` as an expression, you must include `else`. This ensures the result always has a value.

Tip: This works similarly to the ternary operator (*`condition ? value1 : value2`*) in languages like Java or C. However, Rust does not have a ternary operator, but using `if...else` as an expression gives you the same effect.

Don't Mix Types

Note: The value from `if` and `else` must be the same type, like two pieces of text or two numbers (in the example above, both are strings).

When you mix types, like a string and an integer, you'll get an error

(REFER TO EXERCISE-5)

What is match in Rust?

match is used when you have **many possible values** and want to choose what code runs based on them.

Instead of writing:

```
if day == 1 {
} else if day == 2 {
} else if day == 3 {
}
```

Rust gives you something much cleaner: match.

◈ Basic match Example

```
fn main() {
    let day = 4;

    match day {
        1 => println!("Monday"),
        2 => println!("Tuesday"),
        3 => println!("Wednesday"),
        4 => println!("Thursday"),
        5 => println!("Friday"),
        6 => println!("Saturday"),
        7 => println!("Sunday"),
        _ => println!("Invalid day."),
    }
}
```

💡 How This Works

- match day → Rust evaluates day **once**
- It checks each branch from top to bottom
- If it finds a match → that branch runs
- _ is the **default case** (like else)

Since day = 4, it prints:

Thursday

◈ The _ (Default Case)

_ => println!("Invalid day."),

_ means:

“Anything that wasn’t matched above.”

Super useful when you don’t want errors for unexpected values.

◈ Matching Multiple Values (Using |)

You can match several values at once using | (OR operator).

```
fn main() {
    let day = 6;

    match day {
        1 | 2 | 3 | 4 | 5 => println!("Weekday"),
        6 | 7 => println!("Weekend"),
        _ => println!("Invalid day"),
    }
}
```

Here:

- If day is 1–5 → prints “Weekday”
- If day is 6 or 7 → prints “Weekend”

So clean 😭✨

◈ match Can Return a Value 🎯

This is where it becomes really powerful.

match is an **expression**, not just a statement.

That means it can return a value.

```
fn main() {
    let day = 4;

    let result = match day {
        1 => "Monday",
        2 => "Tuesday",
        3 => "Wednesday",
        4 => "Thursday",
        5 => "Friday",
        6 => "Saturday",
        7 => "Sunday",
        _ => "Invalid day.",
    };

    println!("{}", result);
}
```

Now result stores "Thursday".

◈ VERY Important Rule 📥

All branches must return the **same type**.

This is valid:

```
1 => "Monday",
2 => "Tuesday",
```

(All return &str)

❌ This is invalid:

```
1 => "Monday",
2 => 2,
```

Because:

- One branch returns a string
- One returns an integer

Rust does NOT allow mixed types.

◈ match vs if

Use:

- if → when checking conditions (>, <, logical checks)
- match → when comparing a value against multiple exact patterns

Example:

✓ Good for match

```
match grade {
    'A' => ...
    'B' => ...
}
```

✓ Good for if

```
if age > 18 {
}
```

Loops

Loops can execute a block of code as long as a specified condition is reached.

Loops are handy because they save time, reduce errors, and they make code more readable. For example, instead of writing the same line 10 times to print some text, you can use a loop to repeat it for you.

Rust has three types of loops: `loop`, `while`, and `for`.

`loop`

`loop` is the simplest of Rust's three loop types.

It will run forever unless you tell it to stop:

```
loop {
  println!("This will repeat forever!");
}
```

Warning: This loop never stops! You will need to press Ctrl + C to end the program.

To stop a loop, use the break keyword:

```
let mut count = 1;

loop {
  println!("Hello World!");

  if count == 3 {
    break;
  }

  count += 1;
}
```

Example explained:

- This prints "Hello World!" 3 times.
- It uses a counter to keep track of how many times it has looped.
- The counter starts at 1 (let mut count = 1;).
- Each time the loop runs, the counter goes up by 1: (count += 1;).
- When it reaches 3, the loop stops.

Return a Value

You can also return a value from a loop using break with a value.

This lets you save the result of the loop into a variable:

```
let mut count = 1;

let result = loop {
  println!("Hello!");
```

```
  if count == 3 {
    break count; // Stop the loop and return the number 3
  }

  count += 1;
};

println!("The loop stopped at: {}", result);
```

This loop prints "Hello!" until count reaches 3, then stops and returns that number.

Note: When you save the result of a loop into a variable, you must put a semicolon (;) at the end.

Next: Learn how to use while loops to repeat code while a condition is true.

(REFER EXERCISE -6 PROOF)

The `while` Loop

The `while` loop runs as long as a condition is `true`.

False Condition

The while loop checks the condition before each loop, so if the condition is false at the start, the loop will not run at all

Stop a While Loop

You can stop a while loop when you want by using break

Skip a Value

You can skip a value by using the continue statement

(REFER EXERCISE-6)

The `for` Loop

When you know exactly how many times you want to loop through a block of code, use the `for` loop together with the `in` keyword, instead of a `while` loop:

Inclusive Range

If you want to include the last number, use ..= (two dots and an equals sign):

Break and Continue

Just like other loops, you can use break to stop the loop and continue to skip a value

Rust Loops Summary

Rust has three types of loops that let you run code over and over again. Each one is used in different situations:

1. `loop`

The simplest kind of loop. It runs forever unless you stop it with `break`.

```
loop {
  // do something
  if condition {
    break;
  }
}
```

Use `loop` when you don't know in advance how many times to repeat.

---

2. `while`

Repeats code **while a condition is true**. It checks the condition before each loop.

```
while count <= 5 {
  println!("{}", count);
  count += 1;
}
```

Use `while` when you want to repeat code until something happens.

---

3. `for`

Repeats code a fixed number of times.

```
for i in 1..=5 {
  println!("{}", i);
}
```

Use `for` when you know exactly what to loop through.

Extra Keywords

You can use these in any loop:

- `break` - stop the loop
- `continue` - skip a value in the loop

Functions

A function is a block of code that only runs when you call it.

Functions are used to organize your code, avoid repeating yourself, and make your program easier to understand.

Creating a Function

To create a function, use the `fn` keyword, followed by the function name and a set of parentheses `()` and curly braces `{}`:

```
fn function_name() {
  // code to be executed
}
```

Calling a Function

Now that you have created a function, you can execute it by **calling** it.

To call a function, write the name of the function followed by two parantheses `()`.

```
// Create a function
fn say_hello() {
  println!("Hello from a function!");
}

say_hello(); // Call the function
```

Functions with Parameters

You can send information into a function using parameters. Parameters are written inside the parentheses ().

In this example, the function takes a string parameter called name and prints it in the greeting message.

Functions with Return Values

A function can also return a value.

Use the -> symbol in the function header to show what type of value will be returned.

Inside the function, use the return keyword to send the value back:

This function adds two numbers and returns the result.

In Rust, you can omit the return keyword. Just write the value on the last line of the function, **without a semicolon**:

The last line a + b is automatically returned.

Both examples do the same thing. It is up to you which one to use.

Why Use Functions?

- To organize your code
- To avoid repeating the same code
- To make your programs easier to read and change

(REFER EXERCISE-7)

Scope

Now that you understand how functions work, it is important to learn how variables act inside and outside of functions.

Scope refers to where a variable is allowed to be used.

A variable only lives inside the block where it was created. A block is anything inside curly braces `{ }`.

Variable Inside a Function

A variable created inside a function only exists inside that function:

Note: The variable message only exists inside the function. Trying to use it outside the function will cause an error.

Variable Inside a Block

You can also create blocks inside other code, like in if statements or loops. Variables created in these blocks are only valid inside them.

Variables in the Same Scope

In Rust, you **can** declare a new variable with the same name in the same scope using let. This is called **shadowing**:

Example

```
let x = 5;
let x = 10;

println!("x is: {}", x); // prints 10
```

The second x replaces the first one. The value 5 is no longer accessible after the second declaration.

This is different from languages that disallow reusing variable names. In Rust, it's a feature used to transform or update values safely.

You can also reuse a variable name inside a new block:

Example

```
let x = 5;

{
  let x = 10;
  println!("Inside block: {}", x);
}
```

```
println!("Outside block: {}", x);
```

Here, the two x variables are in different scopes. The inner x only exists inside the block. Outside the block, the original value remains.

Note: Even though shadowing is allowed, using the same name too often can make your code harder to read. Use clear names when possible.

Strings

Strings are used to store text.

You have already learned that you can use the `&str` type to create a string

Note that strings are surrounded by double quotes (**"**Hello**"**).

There are two main types of strings in Rust:

- &str - is called "string slices", and is used for fixed text like "Hello"
- String - used when you need a string that can change

In this chapter, you will mostly work with the String type because it is more flexible and can be changed over time.

Create a String

You can create a String from a string literal using the to_string() method or the String::from() function

It is up to you which one to choose - both to_string() and String::from() are very common in Rust.

Change a String

Strings are mutable, so you can change them if they are declared with mut.

Use push_str() to add text to a string:

Use push() to add one character:

Concatenate Strings

You can combine strings using the format! macro:

You can also use the + operator to combine strings, but it can get messy with many values.

String Length

You can use the .len() method to get the length of a string:

(REFER TO EXERCISE-8)

Ownership

Rust uses "ownership" to manage memory in a safe way.

Every value in Rust has an **owner**. The owner is usually a variable.

Ownership Rules

- Each value has one owner
- When the owner goes out of scope, the value is deleted
- You can only have one owner at a time, unless you borrow it (covered in the next chapter)

Basic Ownership Example

In this example, a owns the string. Then we move it to b:

Example

```
let a = String::from("Hello");
let b = a;

// println!("{}", a); Error: a no longer owns the value
println!("{}", b); // Ok: b now owns the value
```

When we assign a to b, the ownership **moves**. This means only b can use the value now, because a is no longer valid.

But simple types like numbers, characters and booleans are **copied**, not moved.

This means you can still use the original variable after assigning it to another:

Example

```
let a = 5;
let b = a;
println!("a = {}", a);  // Works
println!("b = {}", b);  // Works
```

Here, `a` is **copied** into `b`, not moved, so you can still use `b`.

REMOVE ADS

Clone

For other types, like `String`, if you really want to keep the original value and also assign it to another variable, you can use the `.clone()` method, which makes a copy of the data:

Example

```
let a = String::from("Hello");
let b = a.clone(); // Now both have the same value

println!("a = {}", a);  // Works
println!("b = {}", b);  // Works
```

However, if you don't need to own the value twice, using a reference (`&`) is usually better than cloning, which you will learn more about in the next chapter.

Why Ownership Matters

- Rust uses ownership to automatically free memory when it's no longer needed
- It prevents bugs like using memory that's already been deleted
- It is one of the reasons Rust is so safe and fast

Next: Learn about **borrowing** - how to let other parts of your program use a value without taking ownership.

Borrowing and References

Sometimes you want to use a value without taking ownership of it.

Rust lets you do this using a **reference** - this is called **borrowing**:

What is a Reference?

A reference lets you look at a value without owning it. You create a reference using the & symbol:

Example

```
let a = String::from("Hello");
let b = &a;

println!("a = {}", a);
println!("b = {}", b);
```

Since b is only borrowing the value, a still owns it.

Mutable References

If you want to **change** a value through a reference, you need to make the reference mut:

Example

```
let mut name = String::from("John");
let name_ref = &mut name;
name_ref.push_str(" Doe");

println!("{}", name_ref); // John Doe
```

Note: You can only have one mutable reference to a value at a time!

Why Borrowing is Important

Borrowing helps you reuse values safely, without giving them away.

- It lets you use values without taking ownership
- It avoids cloning, which can be slow for large data
- It makes your programs safer and faster

Data Structures

In Rust, data structures are used to store and organize values.

Rust provides many built-in data structures. Each is used to handle data in different ways.

Some of the most common are:

- Array
- Vector (Vec)
- Tuple
- HashMap

We will explore all of them in detail later, but for now, here's a quick introduction to each one.

Arrays

An array in Rust is a fixed-size list of values, all of the same type.

You cannot grow or shrink an array after it's created.

To access an array element, refer to its index number.

Array indexes start with 0: [0] is the first element, [1] is the second element, etc.

Example

```
let fruits = ["apple", "banana", "orange"];
println!("Last fruit: {}", fruits[2]);
```

Vectors

A vector is a resizable array. Unlike regular arrays, vectors can grow or shrink in size.

Example

```
let mut fruits = vec!["apple", "banana"];
fruits.push("cherry");

println!("Last fruit: {}", fruits[2]);
```

Tuples

A tuple can hold multiple values of different types. It is useful when grouping different types together.

You access tuple elements using a dot and an index number, like `person.1`, etc:

Example

```
let person = ("John", 30, true);
println!("Name: {}", person.0);
println!("Age: {}", person.1);
println!("Is active: {}", person.2);
```

HashMaps

A HashMap stores key-value pairs. It lets you look up a value using a key.

To use HashMap, you must import it from the standard library.

Example

```
// Import HashMap
use std::collections::HashMap;

fn main() {
  let mut capitalCities = HashMap::new();
  capitalCities.insert("France", "Paris");
  capitalCities.insert("Japan", "Tokyo");
```

```
  println!("Capital of Japan is {}", capitalCities["Japan"]);
}
```

Data Structure	Use Case	Can Grow?
Array	Fixed-size list of same-type values	No
Vector (Vec)	Growable list of same-type values	Yes
Tuple	Group different types together	No
HashMap	Key-value lookup	Yes

(RERER EXERCISE-9)

Arrays

Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value.

Create an Array

You can create an array using square brackets [ ], and separate the values with commas.

Note: Make sure all values are of the same data type (integers in the example below):

Example

let numbers = [1, 2, 3, 4, 5];

This creates an array with five integers.

Access Array Elements

To access an array element, refer to its index number.

Array indexes start with 0: [0] is the first element. [1] is the second element, etc.

This statement accesses the value of the first element [0] in numbers:

Example

let numbers = [1, 2, 3, 4, 5];
println!("The first number is: {}", numbers[0]);

Change Array Values

To change the value of a specified element, refer to the index number and assign a new value.

Remember to make the array mutable (using the mut keyword)

Array Length

You can get the number of elements in an array using the .len() method

Loop Through an Array

You can loop through the array elements with the for loop.

Print the Entire Array

Note: When printing the whole array, you must use {:?} inside println!

If you are just printing one element from the array, you can use {}.

To Sum Up:

When printing a single element from an array, use {}.

When printing the entire array, use {:?}.

If you are wondering why we didn't use {:?} inside the loop (in the example earlier on this page), it is because in a loop, you are printing each single value from the array. Since each value is a string, {} works. But to print the whole array, you need to use {:?}.

A good rule is to use {} for basic types like strings, numbers, and booleans, and {:?} for data structures like arrays and vectors - when printing the entire structure.

Fixed Size (Arrays) vs. Dynamic Size (Vectors)

You will often hear the terms **fixed size** and **dynamic size** when talking about arrays in Rust.

This is because **arrays in Rust have a fixed size**, meaning you cannot add or remove elements after the array is created:

Example

```
// An array with 3 elements
let mut cars = ["Volvo", "BMW", "Ford"];

// Trying to add another element (a fourth element) to the cars array will result in an error
cars[3] = "Mazda";   // Error: index out of bounds
```

Vectors - Dynamic Size Example

For operations that require adding and removing array elements, you can use **Vectors**, which are resizable arrays.

The size of a vector is dynamic, meaning it can grow and shrink as needed.

You can use the vec! macro to create a vector:

Example

```
// A vector with 3 elements
let mut cars = vec!["Volvo", "BMW", "Ford"];

// Add another element
cars.push("Mazda");

println!("{:?}", cars); // ["Volvo", "BMW", "Ford", "Mazda"]
```

◈ What is a Vector?

A **vector (Vec<T>)** is a **resizable array**.

Unlike normal arrays:

- Arrays → fixed size
- Vectors → can grow and shrink

Vectors are stored on the **heap** and are used when the size is not known at compile time.

◈ Creating a Vector

We use the vec! macro.

It creates a vector and automatically infers the type.

◈ Accessing Elements

- Use index numbers.
- Indexing starts from **0**.
- Same as arrays.

◈ Modifying Elements

To change values:

- The vector must be **mutable (mut)**
- Access using index
- Assign a new value

◈ Adding Elements

Vectors grow at the **end** using:

- push() → adds element to end

◈ Removing Elements

- pop() → removes last element
- remove(index) → removes element at specific index

⚠ Removing from the beginning or middle is slower
(because all elements shift positions)

◈ Inserting Elements

- insert(index, value) → adds at a specific position

◈ Vector Length

- .len() → returns number of elements

◈ Looping Through a Vector

Use:

for item in &vector

Why &?

Because:

- Without & → values are moved (ownership taken)
- With & → values are borrowed (vector still usable later)

This is called **borrowing** in Rust.

Tuples

A **tuple** is a group of values of different types, stored in a single variable.

Tuples are useful when you want to return or work with multiple values together.

Create a Tuple

Tuples are written using parentheses (), with values separated by commas:

Example

```
let person = ("John", 30, true);
```

This tuple contains a &str, an i32, and a bool.

Access Tuple Values

You can access tuple values by using a dot . followed by the index:

Unpack a Tuple

When we create a tuple, we normally assign values to it. This is called "packing" a tuple:

Example

```
let person = ("Jenny", 45, false);
```

But, in Rust, we are also allowed to extract the values back into variables. This is called "unpacking":

Example

```
let person = ("Jenny", 45, false);
let (name, age, active) = person;

println!("Name: {}", name);
println!("Age: {}", age);
println!("Active: {}", active);
```

Return a Tuple from a Function

Tuples are often used to return multiple values from a function:

(EX-9)

HashMap

A **HashMap** is a collection of key/value pairs.

HashMaps are great when you want to store values and find them by a key.

To use HashMap, you must import it from Rust's standard library:

use std::collections::HashMap;

Create a HashMap

You can create a new, empty HashMap and add items to it:

Example

```
// Import HashMap
use std::collections::HashMap;

fn main() {
  // Create a HashMap called capitalCities
  let mut capitalCities = HashMap::new();

  // Add keys and values (Country, City)
  capitalCities.insert("England", "London");
  capitalCities.insert("Germany", "Berlin");
  capitalCities.insert("Norway", "Oslo");

  println!("{:?}", capitalCities);
}
```

Access Values

You can use the .get() method to access a value in a HashMap by its key

Update Values

If you insert a new value using a key that already exists, the old value is replaced with the new one:

Remove Values

To remove a key from a HashMap, use the .remove() method:

Loop Through a HashMap

You can use a for loop to go through all key/value pairs:

Why Use HashMaps?

- To store data by key
- To quickly look up values
- To group related data (like names and scores)

Note: HashMaps require keys to be unique. Inserting the same key again will overwrite the old value.

Structs

A **struct** (short for "structure") is a custom data structure that lets you group related values together.

You can think of a struct like a mini-database for one thing, like a person with a name and age.

Create a Struct

You define a struct using the struct keyword and place the fields (variables) inside:

Example

```
struct Person {
  name: String,
  age: u32,
  can_vote: bool,
}
```

Once you have a struct, you can create an object of it.

Then, you can access the fields of the struct using dot syntax (.):

Change a Field

To change a value inside a struct, you must make the struct object mutable by using mut:

Why Use Structs?

- To group related data in a clean way
- To make your code easier to read and maintain

- To create real-world examples, like users, books, cars, etc.

◈ What is an Enum?

An **enum (enumeration)** is a custom data type that can have **one of several predefined values**.

Each possible value is called a **variant**.

Think of it like saying:

"This variable can only be ONE of these options."

◈ Why Use Enums?

Enums are useful when:

- A value must be one of a fixed set
- You want safer code
- You want better readability
- You want to use match to handle different cases

Examples in real life:

- Directions → Up, Down, Left, Right
- Days → Monday, Tuesday, etc.
- Status → Success, Error
- Traffic light → Red, Yellow, Green

◈ Creating an Enum

Use the enum keyword.

Each variant is separated by commas.

Example idea:

Direction can only be:

- Up
- Down
- Left

- Right

◈ Using an Enum

To use a variant:

EnumName::VariantName

The :: is called the **path separator**.

◈ Enums + match

Enums work perfectly with match.

Rust forces you to handle **all variants** (exhaustive matching).

This makes your program safer — you can't forget a case.

◈ Enums With Data

Enum variants can also store data.

This is VERY powerful.

Example idea:

- Success(message)
- Error(message)

Each variant can hold its own data.

◈ Why Enums Are Powerful in Rust

- Group related values into one type
- Prevent invalid values
- Work perfectly with match
- Can store different types of data per variant

This is one of Rust's strongest features.